



Instituto Tecnológico de Mazatlán



**Buffer Circular con Interrupción Externa
con Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila
Carlos Tadeo Ibarra Castañeda
Dulce Princesa Ulivarría González

1. Introducción

Las interrupciones externas permiten que un microcontrolador reaccione de inmediato a un evento físico, como la pulsación de un botón, sin tener que estar revisando constantemente su estado dentro del loop() principal. Cuando estos eventos pueden ocurrir con mayor rapidez de la que el programa principal es capaz de procesarlos, es común almacenarlos temporalmente en una estructura de datos conocida como buffer circular, que reutiliza un arreglo de tamaño fijo como si fuera una cola continua.

En la presente práctica se implementó un sistema de buffer circular en un Arduino UNO R4 WiFi, en el que una interrupción externa, disparada por un pulsador conectado al pin D2, registra el instante exacto (en microsegundos) de cada evento dentro de un arreglo circular de 32 posiciones, mientras la matriz LED integrada de la placa ejecuta una animación de forma continua e independiente. El objetivo fue verificar que el registro de eventos, el filtro antirrebote dentro de la interrupción y la animación de la matriz funcionaran de manera simultánea y sin bloquearse entre sí.

2. Objetivos

Objetivo general:

- Implementar un buffer circular alimentado por una interrupción externa en un Arduino UNO R4 WiFi, capaz de registrar eventos de un pulsador sin bloquear la animación de la matriz LED integrada.

Objetivos específicos:

- Configurar una interrupción externa en el pin D2, disparada por la pulsación de un botón.
- Implementar un filtro antirrebote de 40 ms dentro de la rutina de interrupción, para evitar registrar una misma pulsación varias veces.
- Almacenar cada evento en un arreglo circular de 32 posiciones junto con su marca de tiempo en microsegundos, obtenida con micros().
- Verificar que la animación de la matriz LED continúe en movimiento mientras se registran los eventos, confirmando que la interrupción no bloquea el resto del programa.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi (con matriz LED integrada)
- Pulsador (pushbutton) conectado al pin digital D2
- Protoboard y cables de conexión (jumpers)
- Arduino IDE 2 (con monitor serie a 9600 baudios)

4. Desarrollo / Metodología

El pulsador se conectó entre el pin digital 2 del Arduino y GND, de forma que cada pulsación pudiera detectarse mediante una interrupción externa asociada a ese pin. No se utilizó ningún componente adicional, ya que la animación se mostró en la matriz LED integrada de la propia placa UNO R4 WiFi.

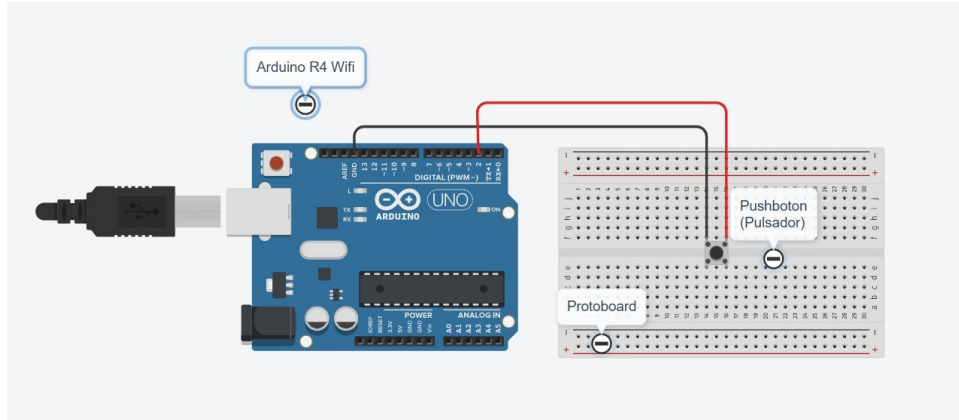
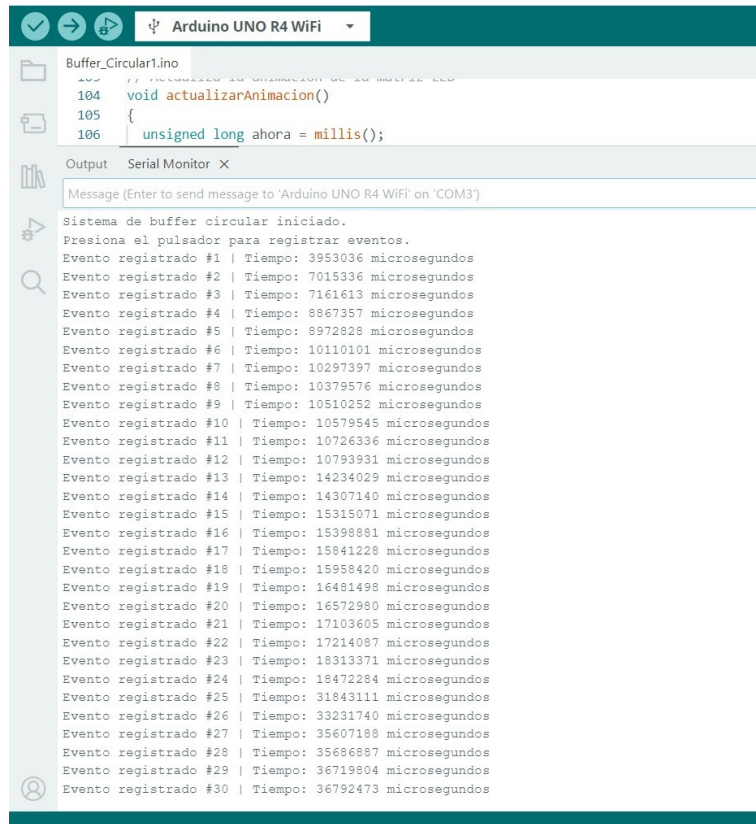


Figura 1. Diagrama esquemático del circuito: Arduino UNO R4 WiFi, protoboard y pulsador conectado al pin D2.

El programa (Buffer_Circular1.ino) define un arreglo circular de 32 posiciones, con un índice de escritura y uno de lectura independientes. Dentro de la rutina de interrupción asociada al pin D2 se implementó un filtro antirrebote de 40 ms: solo se registra un nuevo evento si ha transcurrido al menos ese tiempo desde el último registrado, evitando que un mismo toque físico del botón se cuente como varias pulsaciones. Cada evento válido se guarda junto con el valor de `micros()` en el momento de la interrupción, y el índice de escritura avanza de forma circular, reiniciándose a cero al llegar a la posición 32. De forma independiente, la función `actualizarAnimacion()` utiliza `millis()` para actualizar la animación de la matriz LED en cada iteración del `loop()`, sin depender de si hubo o no un nuevo evento registrado.

5. Evidencia del circuito y del monitor serie funcionando

A continuación se presenta la captura del monitor serie del Arduino IDE 2, mostrando el registro de los eventos capturados al presionar el pulsador repetidamente, cada uno con su marca de tiempo en microsegundos desde el arranque.



The screenshot shows the Arduino IDE interface with the 'Serial Monitor' window open. The code in the background is 'Buffer_Circular1.ino', which includes a function 'actualizarAnimacion()' that uses 'millis()' to track time. The Serial Monitor output displays a series of messages indicating the start of the circular buffer system and subsequent event registrations, each with a timestamp in microseconds.

```
Buffer_Circular1.ino
// ...
104 void actualizarAnimacion()
105 {
106   unsigned long ahora = millis();

Output Serial Monitor X
Message (Enter to send message to 'Arduino UNO R4 WiFi' on 'COM3')

Sistema de buffer circular iniciado.
Presiona el pulsador para registrar eventos.
Evento registrado #1 | Tiempo: 3953036 microsegundos
Evento registrado #2 | Tiempo: 7015336 microsegundos
Evento registrado #3 | Tiempo: 7161613 microsegundos
Evento registrado #4 | Tiempo: 8867357 microsegundos
Evento registrado #5 | Tiempo: 8972828 microsegundos
Evento registrado #6 | Tiempo: 10110101 microsegundos
Evento registrado #7 | Tiempo: 10297397 microsegundos
Evento registrado #8 | Tiempo: 10379576 microsegundos
Evento registrado #9 | Tiempo: 10510252 microsegundos
Evento registrado #10 | Tiempo: 10579545 microsegundos
Evento registrado #11 | Tiempo: 10726336 microsegundos
Evento registrado #12 | Tiempo: 10793931 microsegundos
Evento registrado #13 | Tiempo: 14234029 microsegundos
Evento registrado #14 | Tiempo: 14307140 microsegundos
Evento registrado #15 | Tiempo: 15315071 microsegundos
Evento registrado #16 | Tiempo: 15398881 microsegundos
Evento registrado #17 | Tiempo: 15841228 microsegundos
Evento registrado #18 | Tiempo: 15958420 microsegundos
Evento registrado #19 | Tiempo: 16481498 microsegundos
Evento registrado #20 | Tiempo: 16572980 microsegundos
Evento registrado #21 | Tiempo: 17103605 microsegundos
Evento registrado #22 | Tiempo: 17214087 microsegundos
Evento registrado #23 | Tiempo: 18313371 microsegundos
Evento registrado #24 | Tiempo: 18472284 microsegundos
Evento registrado #25 | Tiempo: 31843111 microsegundos
Evento registrado #26 | Tiempo: 33231740 microsegundos
Evento registrado #27 | Tiempo: 35607188 microsegundos
Evento registrado #28 | Tiempo: 35686887 microsegundos
Evento registrado #29 | Tiempo: 36719804 microsegundos
Evento registrado #30 | Tiempo: 36792473 microsegundos
```

Figura 2. Monitor serie del Arduino IDE 2: eventos registrados por el buffer circular, cada uno con su tiempo en microsegundos.

Asimismo, se presenta la evidencia fotográfica del circuito físico armado, con el pulsador conectado al Arduino UNO R4 WiFi y la matriz LED integrada mostrando su animación durante las pruebas.

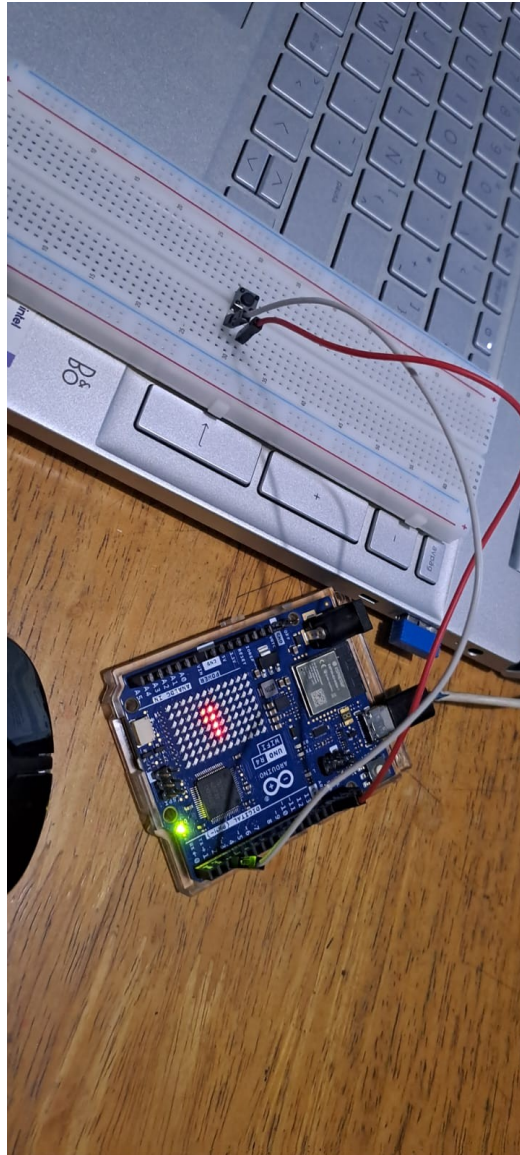


Figura 3. Pulsador conectado al pin D2 del Arduino UNO R4 WiFi, con la matriz LED integrada en movimiento.

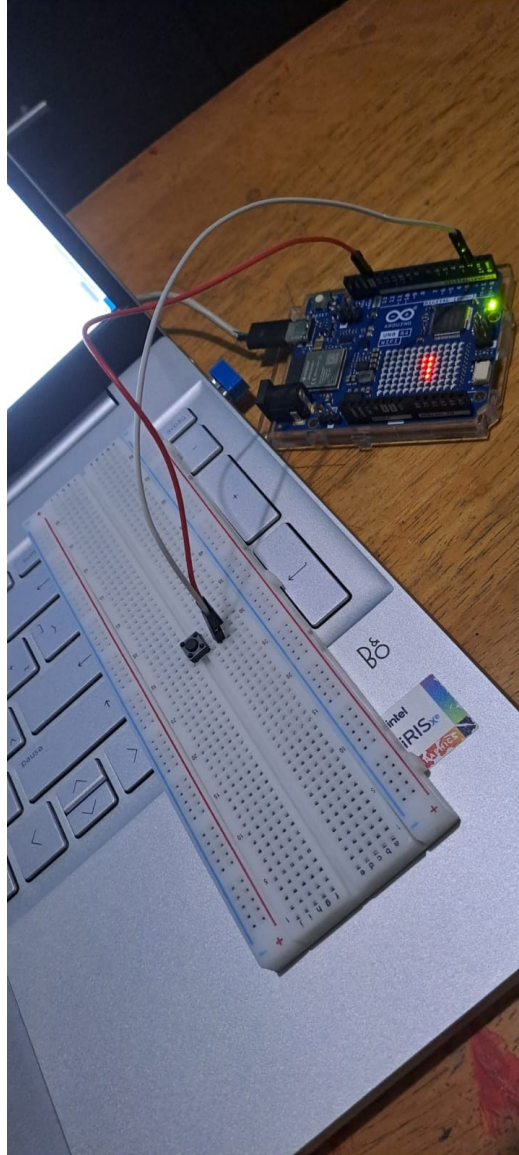


Figura 4. Vista general del circuito armado durante las pruebas de la práctica.

6. Análisis de resultados

Las pruebas se realizaron el 15 de septiembre de 2026, presionando el pulsador conectado al pin D2 repetidamente mientras la matriz LED integrada ejecutaba su animación. El Monitor Serial registró 30 eventos consecutivos, del #1 al #30, cada uno con su tiempo en microsegundos desde el arranque. Las gráficas y tablas de esta sección se calcularon a partir de esos 30 tiempos.

La Figura 5 muestra el avance del conteo de eventos a lo largo de la prueba; la Figura 6 muestra el tiempo transcurrido entre cada evento y el anterior. Ambas se construyeron únicamente con los 30 tiempos capturados en el monitor serie; no representan el número de espacios ocupados dentro del buffer en cada instante.

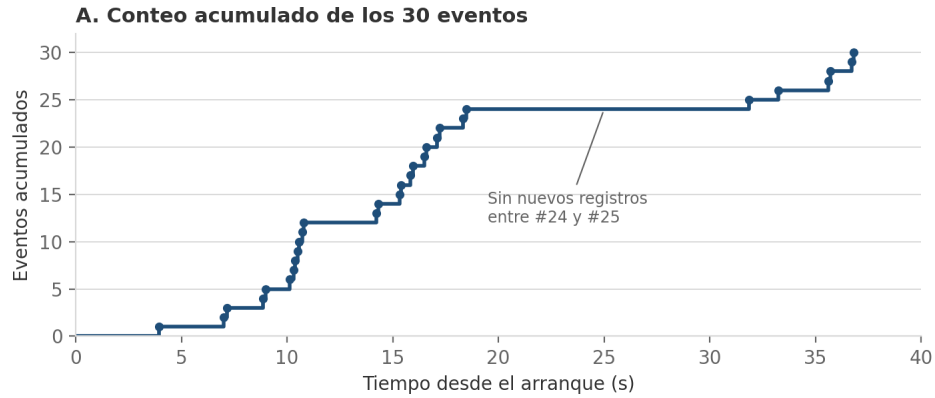


Figura 5. Conteo acumulado de los 30 eventos registrados. El tramo horizontal entre los eventos #24 y #25 corresponde a un periodo sin nuevos registros.

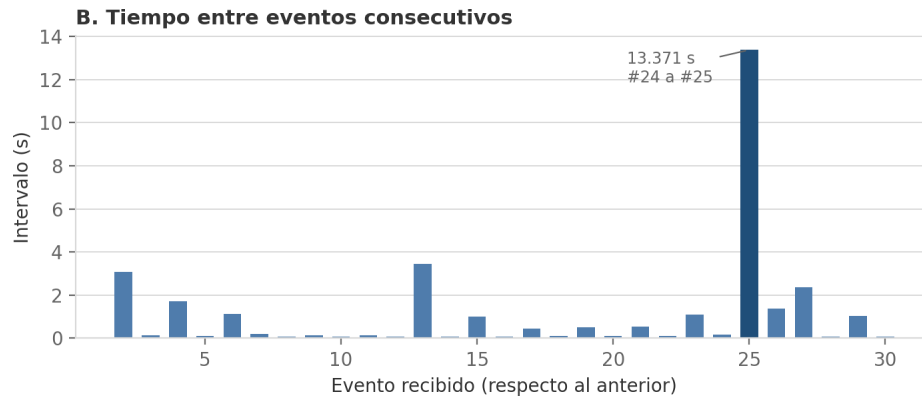


Figura 6. Tiempo entre cada evento y el anterior. El intervalo más largo (13.371 s) ocurrió entre los eventos #24 y #25; todos los intervalos superan ampliamente el umbral de 40 ms del filtro antirrebote.

La Tabla I resume los indicadores calculados a partir de los 30 tiempos registrados. La Tabla II muestra los primeros diez eventos de la prueba completa, con su tiempo en microsegundos y el intervalo respecto al evento anterior.

Tabla I. Indicadores calculados a partir de los 30 eventos registrados.

Indicador calculado	Resultado
Eventos registrados	30 (del #1 al #30)
Lapso entre el primer y el último evento	32.839437 s
Menor intervalo entre eventos	67.595 ms, entre #11 y #12
Mayor intervalo entre eventos	13.370827 s, entre #24 y #25
Mediana de los 29 intervalos	158.913 ms
Mensajes de “buffer lleno” observados	Ninguno en la porción visible del registro

Tabla II. Primeros diez eventos de la prueba.

Evento	Tiempo (microsegundos)	Intervalo desde el anterior (microsegundos)
#1	3,953,036	—
#2	7,015,336	3,062,300
#3	7,161,613	146,277
#4	8,867,357	1,705,744
#5	8,972,828	105,471
#6	10,110,101	1,137,273
#7	10,297,397	187,296
#8	10,379,576	82,179
#9	10,510,252	130,676
#10	10,579,545	69,293

Observaciones sobre el comportamiento del sistema:

- El buffer circular respetó el orden de llegada de los eventos: los 30 tiempos registrados son estrictamente crecientes, sin repeticiones ni saltos hacia atrás.
- El filtro antirrebote (40 ms) dentro de la interrupción funcionó según lo esperado: el intervalo más corto observado entre dos eventos fue de 67.595 ms, mayor al umbral de 40 ms, por lo que no se detectaron rebotes registrados como pulsaciones independientes.
- No apareció ningún mensaje de eventos rechazados por buffer lleno durante la prueba, lo que indica que el loop() retiró los eventos del buffer con suficiente rapidez para no llenar las 32 posiciones disponibles con solo 30 pulsaciones.
- La animación de la matriz LED se mantuvo en movimiento durante toda la prueba, incluso mientras se presionaba el pulsador repetidamente, lo que confirma que la interrupción no bloqueó el resto del programa.
- La prueba no llegó a las 32 posiciones del buffer ni provocó que los índices de escritura y lectura regresaran al inicio del arreglo; comprobar la vuelta completa del buffer y su respuesta al llenarse requeriría una prueba aparte con más de 32 pulsaciones sin que el loop() las procese.

7. Conclusiones individuales

Ximena

Esta práctica me permitió comprender cómo una interrupción externa puede capturar eventos de manera confiable sin depender de que el loop() esté revisando el pin en ese preciso instante. Ver que los 30 tiempos registrados fueran estrictamente crecientes, sin saltos ni repeticiones, me dio confianza en que la lógica del buffer circular y del filtro antirrebote estaban funcionando correctamente.

Me pareció interesante que, aunque presionamos el botón de forma irregular, con pausas largas como la de 13.371 s entre los eventos #24 y #25, el sistema nunca perdió el orden ni mezcló los datos. Esto me hizo entender que la fortaleza de un buffer circular no está en la velocidad con la que se llena, sino en que siempre respeta el orden de llegada sin importar cuán espaciados estén los eventos.

Carlos Tadeo

El desarrollo de esta práctica reforzó mi comprensión sobre la diferencia entre atender un evento de forma inmediata, dentro de una interrupción, y procesarlo después, dentro del `loop()` principal. Implementar el filtro antirrebote directamente en la rutina de interrupción, comparando el tiempo transcurrido contra un umbral de 40 ms, me ayudó a ver cómo se puede proteger un sistema de eventos falsos sin necesidad de detener ninguna otra tarea del programa.

También me pareció relevante que, en las 30 pulsaciones registradas, nunca se llegó a ocupar la totalidad de las 32 posiciones del buffer ni apareció ningún mensaje de "buffer lleno". Esto nos dejó pendiente comprobar qué sucede cuando el buffer realmente se llena y los índices de escritura y lectura deben dar la vuelta completa al arreglo, algo que consideramos importante para una prueba futura más exigente.

Dulce Princesa

Considero que el mayor aprendizaje de esta práctica fue ver, de forma muy clara, que dos tareas aparentemente independientes —la animación de la matriz LED y el registro de eventos del pulsador— pueden ejecutarse al mismo tiempo sin interferir entre sí, siempre que ninguna de las dos bloquee la ejecución del programa. Confirmar que la matriz nunca se detuvo, ni siquiera durante las pulsaciones más seguidas, fue una buena evidencia de que la interrupción estaba bien diseñada.

Adicionalmente, analizar la Tabla II y notar la gran diferencia entre el intervalo más corto (67.595 ms) y el más largo (13.370827 s) me ayudó a entender que un buffer circular debe estar preparado para recibir eventos a ritmos muy distintos, desde pulsaciones casi seguidas hasta pausas de varios segundos, y que precisamente por eso tiene sentido usar una estructura de tamaño fijo que se reutiliza en lugar de una lista que crece sin límite.

Referencias

[1] Arduino, "Arduino UNO R4 WiFi", documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc>

[2] Arduino, "attachInterrupt()", referencia oficial del lenguaje Arduino. [En línea]. Disponible en: <https://docs.arduino.cc/language-reference/en/functions/external-interrupts/attachInterrupt>